Guía Técnica de Configuración y Puesta en Marcha Local
Proyecto: Conecta MINEDUC (SSO + Sistema de Notas)

1. Arquitectura General del Entorno y Fronteras de Aislamiento
    1.1. Análisis Crítico de Aislamiento de Red y Riesgos Remotos
2. Prerrequisitos del Sistema
    Procedimiento Oficial de Instalación de Prerrequisitos
3. Clonado del Repositorio e Instalación de Dependencias
    3.1. Clonar el repositorio y verificar la rama
    3.2. Instalación de dependencias
4. Variables de Entorno e Inventario Técnico
    4.1. Inventario Detallado de Variables
5. Firebase Emulator Suite (Auth y Firestore)
    5.1. Comandos de Ejecución
    5.2. Servicios y Puertos
6. Mailpit: Captura Local de Correos OTP (2FA)
7. Datos Iniciales y Scripts de Inicialización (Scratch)
    7.1. Inspección de `scratch/seed-superadmin.mjs`
    7.2. Otros Scripts en `scratch/`
8. Orden de Arranque de los Servicios Locales
9. Rutas de Entrada y Flujos Funcionales Verificados
    9.1. Puntos de Entrada
    9.2. Mecanismos de Autenticación y Autorización Verificados en Código
10. Verificaciones de Calidad y Pruebas (Auditoría del 14/10/2026)
    10.1. Tipado Estático (TypeScript)
    10.2. Pruebas Unitarias (Jest)
    10.3. Linter (ESLint)
11. Estructura del Repositorio
12. Solución de Problemas Frecuentes (Troubleshooting)
13. Checklist de Primera Ejecución para Nuevos Desarrolladores

Guía Técnica de Configuración y Puesta en Marcha Local

Proyecto: Conecta MINEDUC (SSO + Sistema de Notas)

> **Documento de Auditoría y Puesta en Marcha Técnica**
> **Fecha de última auditoría y verificación:** 14 de octubre de 2026
> **Estado del repositorio:** Inspeccionado sobre rama de desarrollo activa

Este documento detalla el procedimiento técnico, verificado e integral para configurar, inicializar y operar localmente la plataforma **Conecta MINEDUC (SSO)** y el subsistema de **Notas**. Está redactado para que un desarrollador nuevo con conocimientos en Next.js, React, TypeScript y Firebase pueda preparar su entorno de manera reproducible, segura y comprendiendo el comportamiento real del código.

---

1. Arquitectura General del Entorno y Fronteras de Aislamiento

El sistema opera bajo un patrón **BFF (Backend-For-Frontend)** integrado en Next.js:

- **Frontend:** Next.js 16.2 (App Router con Turbopack), React 19.2, TypeScript 5.9, Tailwind CSS v4, Radix UI y Lucide Icons.
- **Autenticación en Cliente:** Firebase Client SDK (`firebase` v12.19.0) enrutado localmente al emulador de Firebase Auth.
- **Capa BFF / APIs de Servidor:** Route Handlers en `src/app/api/` que consumen Firebase Admin SDK (`firebase-admin` v14.5.0) en Node.js, conectado a los emuladores locales de Auth y Cloud Firestore.
- **Persistencia de Base de Datos:** Cloud Firestore emulado en memoria (puerto `8080`), con colecciones principales `usuarios` , `usuarios_indices` , `auth_otps` y `notas_otps` .
- **Servicio de Mensajería y Correo:** `nodemailer` configurado hacia el servidor SMTP local **Mailpit** (puerto `1025`) con panel de inspección web (puerto `8025`).

```
flowchart TD
    subgraph Browser ["Navegador Web (Desarrollador)"]
        UI["Next.js UI (App Router)"]
        ClientAuth["Firebase Client SDK (Auth)"]
    end

    subgraph LocalServices ["Servicios en Host Local (127.0.0.1)"]
        NextServer["Next.js Server / BFF (Puerto 3000)"]
        AdminSDK["Firebase Admin SDK (Node.js)"]
        AuthEmulator["Firebase Auth Emulator (Puerto 9099)"]
        FirestoreEmulator["Firestore Emulator (Puerto 8080)"]
        EmulatorUI["Firebase Emulator Suite UI (Puerto 4000)"]
        MailpitSMTP["Mailpit SMTP (Puerto 1025)"]
        MailpitUI["Mailpit Web UI (Puerto 8025)"]
    end

    subgraph ExternalRisks ["Servicios Remotos / Fronteras Externas"]
        GCS["Google Cloud Storage API (Sin Emulador)"]
        ExtAssets["CDNs Externos (ui-avatars, YouTube, shadcn)"]
        ExtFonts["Google Fonts API (css2)"]
    end

    UI -->|Renderizado / Peticiones HTTP| NextServer
    ClientAuth -->|signInWithEmailAndPassword| AuthEmulator
    NextServer -->|server-only| AdminSDK
    AdminSDK -->|verifyIdToken / setCustomUserClaims| AuthEmulator
    AdminSDK -->|Lectura / Escritura Documentos| FirestoreEmulator
    NextServer -->|nodemailer (Envío OTP)| MailpitSMTP
    AuthEmulator -.->|Inspección| EmulatorUI
    FirestoreEmulator -.->|Inspección| EmulatorUI
    MailpitSMTP -.->|Inspección de Correos| MailpitUI

    %% Fronteras y riesgos
    NextServer -.->|src/lib/gcs.ts (Invocación directa si se
importa)| GCS
    UI -.->|Carga de imágenes remotas| ExtAssets
    NextServer -.->|src/lib/google-fonts.ts| ExtFonts
```

1.1. Análisis Crítico de Aislamiento de Red y Riesgos Remotos

> [!WARNING] **No asumir aislamiento hermético total únicamente por usar el ID `demo-sso`:** El prefijo `demo-` en Firebase CLI garantiza que los SDKs de **Firebase Auth** y **Cloud Firestore** no envíen tráfico a la nube de Google. Sin embargo, existen dependencias y archivos en el proyecto que escapan a este aislamiento:
>
> 1. **Google Cloud Storage (`@google-cloud/storage`):** `firebase.json` **NO** tiene emulador de Cloud Storage configurado. El archivo `src/lib/gcs.ts` instancia `new Storage()`. Aunque este archivo no está importado actualmente en las rutas activas de SSO ni Notas, cualquier componente o endpoint que lo invoque intentará contactar

`storage.googleapis.com` mediante credenciales ADC de Google Cloud.

2. **Imágenes y Recursos Remotos:** `next.config.ts` autoriza la carga de imágenes externas desde `i.pravatar.cc`, `img.youtube.com`, `i.ytimg.com` y `storage.googleapis.com`. Asimismo, `scratch/seed-superadmin.mjs` asigna URLs a `https://ui-avatars.com` y `auth-context.tsx` utiliza `https://github.com/shadcn.png`. Su renderizado genera llamadas HTTP salientes a internet.
3. **Google Fonts:** El módulo auxiliar `src/lib/google-fonts.ts` contiene funciones para consultar `https://fonts.googleapis.com/css2`.
4. **Google OAuth:** Si la variable `NEXT_PUBLIC_FIREBASE_AUTH_EMULATOR_HOST` no está presente en `.env.local`, el botón "Iniciar sesión con Google" abrirá un flujo real contra `accounts.google.com` y Firebase de producción.
5. **SMTP:** Nodemailer solo se mantiene local si `SMTP_HOST` apunta a `127.0.0.1` (Mailpit). Si se configuran credenciales externas (SendGrid/Gmail), los correos saldrán a internet.

---

2. Prerrequisitos del Sistema

Herramientas necesarias verificadas en entorno Windows 10/11 (compatibles con Linux/macOS):

Herramienta	Versión Verificada	Propósito en el Proyecto	Comando de Verificación
Node.js	`v22.14.0` (LTS 22.x)	Ejecución del servidor Next.js, BFF y scripts Node.	`node -v`
npm	`10.9.2`	Gestor de paquetes oficial (`package-lock.json`).	`npm -v`
Git	`>= 2.30.0`	Control de versiones del código fuente.	`git --version`
Java (JDK)	`OpenJDK 21.0.12.1 LTS` (o `>= 11`)	Motor de ejecución obligatorio para Cloud Firestore Emulator.	`java -version`

Herramienta	Versión Verificada	Propósito en el Proyecto	Comando de Verificación
Firebase CLI	`15.33.0`	Orquestación de emuladores locales (`firebase-tools`).	`firebase --version`
Mailpit	`v1.31.4`	Servidor SMTP local para captura de correos OTP (2FA).	`mailpit version`

Procedimiento Oficial de Instalación de Prerrequisitos

1. **Node.js 22 LTS y npm:**
   - Windows (winget): `winget install OpenJS.NodeJS.LTS`
   - O descargar desde nodejs.org.
2. **Java JDK 21 LTS:**
   - Windows (winget): `winget install Microsoft.OpenJDK.21`
   - Requiere que la variable `JAVA_HOME` o el ejecutable `java` estén en el `PATH` .
3. **Firebase CLI:**
   - Instalar globalmente: `npm install -g firebase-tools`
   - Para proyectos de prueba `demo-*` no se requiere iniciar sesión, pero puedes ejecutar `firebase login` si dispones de cuenta Google.
4. **Mailpit:**
   - Windows (winget): `winget install axllent.mailpit`
   - Windows (scoop): `scoop install mailpit`
   - O descargar el binario desde GitHub Releases de Mailpit.

---

3. Clonado del Repositorio e Instalación de Dependencias

3.1. Clonar el repositorio y verificar la rama

```
git clone https://github.com/JeimyMateus/SSO.git
cd SSO
git checkout devJeimy   # O la rama de trabajo asignada (ej. main)
```

3.2. Instalación de dependencias

El proyecto utiliza **React 19** y **Next.js 16**. Ciertas dependencias del ecosistema aún declaran peer dependencies de React 18, por lo que **es indispensable usar el flag** `--legacy-peer-deps`:

- **Para instalación estándar de desarrollo:**

```
npm install --legacy-peer-deps
```

- **Para instalación limpia y reproducible (verificada contra** `package-lock.json`**):**

```
npm ci --legacy-peer-deps
```

> [!NOTE] La compatibilidad de `npm ci --legacy-peer-deps` fue validada con éxito frente al lockfile actual (`exit code 0`, 0 errores).

---

4. Variables de Entorno e Inventario Técnico

El archivo `.env.example` en la raíz define la plantilla base. Para configurar el entorno local:

```
Copy-Item .env.example .env.local
```

4.1. Inventario Detallado de Variables

Variable	Ámbito	Estado	Valor L
`NEXT_PUBLIC_FIREBASE_PROJECT_ID`	Cliente / Servidor	**Obligatoria**	`"demo`
`NEXT_PUBLIC_FIREBASE_API_KEY`	Cliente	**Obligatoria**	`"demo`
`NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN`	Cliente	**Obligatoria**	`"demo` `sso.f`

Variable	Ámbito	Estado	Valor L
`NEXT_PUBLIC_FIREBASE_AUTH_EMULATOR_HOST`	Cliente	**CRÍTICA**	`"127.`
`NEXT_PUBLIC_BASE_PATH`	Cliente	Opcional	`""`
`FIREBASE_AUTH_EMULATOR_HOST`	Servidor	**Obligatoria**	`"127.`
`FIRESTORE_EMULATOR_HOST`	Servidor	**Obligatoria**	`"127.`
`SMTP_HOST`	Servidor	**Obligatoria**	`"127.`
`SMTP_PORT`	Servidor	**Obligatoria**	`"1025`
`SMTP_SECURE`	Servidor	**Obligatoria**	`"fals`
`EMAIL_FROM`	Servidor	**Obligatoria**	`"Cone` `reply(`
`SMTP_USER` / `SMTP_PASS`	Servidor	Opcional	*(Vacío*

Variable	Ámbito	Estado	Valor L
`KIT_ASSETS_BUCKET`	Servidor	Opcional	`"mine`
`NEXT_PUBLIC_API_URL_CONTACTO`	Cliente	**OBSOLETA**	*(No rec*

---

5. Firebase Emulator Suite (Auth y Firestore)

Configurado en `firebase.json` bajo modo de proyecto único (`singleProjectMode: true`):

```
{
  "emulators": {
    "auth": { "port": 9099 },
    "firestore": { "port": 8080 },
    "ui": { "enabled": true },
    "singleProjectMode": true
  }
}
```

5.1. Comandos de Ejecución

- **Arranque estándar (en memoria volátil):**

```
npm run emulators
# Comando directo equivalente:
firebase emulators:start --project demo-sso
```

- **Arranque con persistencia de datos entre reinicios:**

```
firebase emulators:start --project demo-sso --import=./emulator-
        data --export-on-exit
```

(Crea y sincroniza la carpeta `emulator-data/` al cerrar el emulador con `Ctrl + C`).

5.2. Servicios y Puertos

Servicio	Puerto	Endpoint / URL	Función
Emulator UI	`4000`	http://localhost:4000	Panel web interactivo para explorar usuarios de Auth y documentos de Firestore.
Firebase Auth	`9099`	`http://127.0.0.1:9099`	Emulador de autenticación (usuarios, tokens, custom claims).
Cloud Firestore	`8080`	`http://127.0.0.1:8080`	Base de datos NoSQL documental local.

---

6. Mailpit: Captura Local de Correos OTP (2FA)

Mailpit intercepta los mensajes enviados por `nodemailer` evitando que salgan a la red.

1. **Iniciar el servicio:**

```
mailpit
```

2. **Puertos configurados:**
   - **Recepción SMTP:** `127.0.0.1:1025`
   - **Bandeja de entrada Web:** http://localhost:8025
3. **Flujo de verificación:** Cuando inicies sesión con contraseña en SSO o en Notas, el sistema despachará un código OTP de 6 dígitos. Abre http://localhost:8025 para consultarlo de inmediato.

---

7. Datos Iniciales y Scripts de Inicialización (Scratch)

7.1. Inspección de `scratch/seed-superadmin.mjs`

El script oficial para poblar el entorno local es `scratch/seed-superadmin.mjs`.

- **Idempotencia: SÍ es idempotente**. Verifica previamente si el usuario existe en Firebase Auth (`getUserByEmail`) y en Firestore (`where("email", "==", email)`). Si ya existe, actualiza sus datos mediante `{ merge: true }`; si no, lo crea. Puede ejecutarse múltiples veces de forma segura.
- **Prerrequisito estricto:** Requiere que los emuladores de Firebase estén levantados en los puertos `9099` y `8080`, ya que se conecta explícitamente a `127.0.0.1:8080` y `127.0.0.1:9099`.
- **Datos generados:**
  - Crea el usuario en Firebase Auth y le asigna los Custom Claims: `{ admin: true, twoFactorVerified: true, debeCambiarPassword: false }`.
  - Crea el documento en la colección `usuarios` de Firestore con estado `Activo`, sede `Planta Central` y asignaciones con permisos completos (`ALL-SEC`) para 9 aplicaciones: SSO, Gestión Docente, SIGE, Geoportal, SAE, SGC, Trámites, Innovación y Notas.
  - Registra los índices atómicos en la colección `usuarios_indices`.

> [!IMPORTANT] **Aviso sobre credenciales del script:** En el código actual de `scratch/seed-superadmin.mjs`, los valores de acceso están definidos en las líneas 20 a 24 del archivo. Para consultar las credenciales de prueba configuradas o definir tus propias credenciales de desarrollo, abre `scratch/seed-superadmin.mjs` antes de ejecutarlo.
> *(Ejemplo ilustrativo seguro:* `admin.sso@mineduc.gob.ec` */* `DemoAdmin2026!*` *).*

- **Ejecutar el sembrado:**

```
node scratch/seed-superadmin.mjs
```

7.2. Otros Scripts en `scratch/`

- `scratch/test-auth-password-flow.mjs`: Script de prueba integral que valida la creación de usuarios con contraseña temporal, el forzado del diálogo `must-change-password` y la actualización definitiva de contraseña. *(Idempotente en ejecución, genera usuarios temporales con timestamp).*
- `scratch/test-crud.mjs`: Script de prueba del CRUD de usuarios. **No es idempotente** en ejecuciones repetidas sobre la misma instancia en memoria

porque intenta insertar un documento con cédula fija ya indexada, arrojando error `409 Conflict`.

---

8. Orden de Arranque de los Servicios Locales

Para un inicio ordenado y sin condiciones de carrera, utiliza terminales independientes:

Paso	Servicio	Comando	Directorio	Puerto / URL
1	Firebase Emulators	`npm run emulators`	Raíz del proyecto	Auth: `9099`, Firestore: `8080`, UI: http://localhost:4000
2	Mailpit SMTP	`mailpit`	Cualquier ubicación	SMTP: `1025`, Web UI: http://localhost:8025
3	Sembrado inicial *(si el emulador está vacío)*	`node scratch/seed-superadmin.mjs`	Raíz del proyecto	N/A (se conecta a 9099 y 8080)
4	Servidor Next.js	`npm run dev`	Raíz del proyecto	http://localhost:3000

---

9. Rutas de Entrada y Flujos Funcionales Verificados

9.1. Puntos de Entrada

- **Raíz (`/`):** Redirige a `/login-sso/`.
- **Login General SSO (`/login-sso`):** Formulario para administradores y funcionarios.
- **Doble Factor SSO (`/doble-factor`):** Validación de código de 6 dígitos. Al verificarse en `POST /api/auth/verify-otp`, se emite el Custom Claim `twoFactorVerified: true`.
- **Login Subsistema de Notas (`/login-notas`):** Acceso para coordinadores y directores de calificaciones.
- **Doble Factor de Notas (`/login-notas/verificar-codigo`):** Valida contra la colección `notas_otps` y exige que el usuario tenga asignaciones activas para `"notas"` en Firestore. Al verificarse en `POST /api/notas/auth/verify-otp`, se emite el Custom Claim `notasTwoFactorVerified: true`.

- **Espacios de Usuario según Perfil:**
  - `/dashboard`: Panel de administración central (Super Admin / Administradores).
  - `/inicio`: Espacio personal para perfil Docente.
  - `/mi-dashboard`: Espacio personal para Usuario Estándar.
  - `/mis-aplicaciones`: Cuadrícula con las aplicaciones activas asignadas.
  - `/uikit`: Catálogo interactivo de componentes del Design System.

9.2. Mecanismos de Autenticación y Autorización Verificados en Código

1. **Google OAuth vs Contraseña:**
   - En `src/lib/auth/server-auth.ts`, los usuarios que inician sesión con `google.com` (Google Workspace) omiten el doble factor OTP local, ya que Google gestiona el MFA a nivel corporativo.
   - Los usuarios autenticados mediante `password` requieren estrictamente completar el flujo 2FA vía OTP.

2. **Validación de Usuario Activo:**
   - La función `verifyActiveUser` consulta Firestore y exige `usuario.active === true` (o estado `Activo`). Si el usuario fue desactivado, el BFF rechaza la petición con código `403 USER_INACTIVE`.

3. **Control de Acceso por Roles (RBAC):**
   - `verifyAdminUser` valida permisos específicos en la aplicación SSO: `ALL-SEC`, `USR-CONS` o roles reconocidos (`ra-sso-sa`, `ra-sso-gz`, etc.).
   - En el cliente, `PortalRouteGuard` bloquea rutas administrativas (`/gestion-usuarios`, `/roles`, `/auditoria`, etc.) a usuarios estándar o docentes, redirigiéndolos automáticamente.

4. **Garantía Atómica en Transacciones:**
   - Cloud Firestore implementa transacciones atómicas en `createUsuario` y `updateUsuario` para garantizar unicidad de correo y documento de identidad mediante la colección `usuarios_indices` (todas las lecturas ocurren antes de las escrituras).
   - **Limitación técnica:** No existe transacción distribuida (2PC) entre Firebase Auth y Firestore. Si la creación de usuario en Auth tiene éxito pero la transacción de Firestore falla, las operaciones se gestionan mediante bloques `try/catch` de compensación.

---

10. Verificaciones de Calidad y Pruebas (Auditoría del 14/10/2026)

Resultados reales observados al ejecutar las herramientas de análisis sobre el código actual:

10.1. Tipado Estático (TypeScript)

```
npx tsc --noEmit
```

- **Resultado: 0 errores** (Código de salida `0`). El tipado estático del proyecto es consistente.

10.2. Pruebas Unitarias (Jest)

> [!NOTE] `package.json` no contiene el script `"test": "jest"`. La ejecución debe realizarse mediante `npx jest`.

```
npx jest
```

- **Resultado observado:**
  - **Suites totales:** 22
  - **Suites pasadas:** 20
  - **Suites falladas:** 2 (`src/modules/gestion-usuarios/components/__tests__/usuario-modal-form.test.tsx`)
  - **Pruebas unitarias:** 148 aprobadas, 3 falladas (debido a discrepancias en el texto esperado de placeholders de formulario).

10.3. Linter (ESLint)

```
npm run lint
```

- **Resultado observado: 14 errores y 427 advertencias** (Código de salida `1`).
  - Error bloqueante: `update-tables.js` en la raíz utiliza `require()` (`@typescript-eslint/no-require-imports`).
  - Advertencias: Variables e importaciones no utilizadas en componentes del módulo `src/modules/uikit/` (`@typescript-eslint/no-unused-vars`). *(Existe un archivo de registro histórico en la raíz: `lint.txt`).*

---

11. Estructura del Repositorio

```
SSO/
├── .firebaserc                     # Proyecto Firebase por defecto
(demo-sso)
├── .gitignore                      # Exclusiones de Git (.env*,
node_modules, .next)
├── Dockerfile                      # Contenedor de producción multi-
stage (node:22-slim)
├── DEVELOPMENT_SETUP.md            # Esta guía maestra de desarrollo
local
├── README.md                       # Documentación institucional del
proyecto
├── cloudbuild.yaml                 # Pipeline de CI/CD para Google
Cloud Build
├── dev.yaml / qa.yaml / prod.yaml  # Despliegues para App Engine
├── eslint.config.mjs               # Configuración de ESLint flat
config
├── firebase.json                   # Definición de emuladores (Auth:
9099, Firestore: 8080, UI: 4000)
├── jest.config.mjs                 # Configuración de pruebas
unitarias Jest
├── next.config.ts                  # Configuración de Next.js
(basePath, next-intl, imágenes)
├── package.json                    # Dependencias y scripts npm
├── package-lock.json               # Árbol de dependencias bloqueado
├── tsconfig.json                   # Configuración del compilador
TypeScript
├── docs/                           # Documentación auxiliar de
Firebase y Email
├── public/                         # Recursos estáticos (logos,
SVGs)
├── scratch/                        # Scripts Node.js de
inicialización y validación
│   ├── seed-superadmin.mjs         # Sembrado idempotente del Super
Admin en emuladores
│   ├── test-auth-password-flow.mjs # Validación de flujo de cambio
forzado de contraseña
│   └── test-crud.mjs               # Prueba automatizada de API de
usuarios
└── src/
    ├── app/                        # Next.js App Router (Páginas y
Route Handlers)
    │   ├── (portal)/               # Vistas protegidas con barra
lateral y Route Guard
    │   ├── api/                    # BFF Route Handlers (auth,
notas, usuarios)
    │   ├── doble-factor/           # Verificación 2FA general SSO
    │   ├── login-notas/            # Acceso y 2FA exclusivo de Notas
    │   ├── login-sso/              # Acceso general institucional
    │   └── uikit/                  # Showcase de componentes del
Design System
    ├── components/                 # UI Kit institucional (Radix UI
/ Shadcn)
```

```
    ├── context/                  # AuthContext (soporte para
Firebase Auth y Mock Users)
    ├── hooks/                    # Custom Hooks (useAuth, etc.)
    ├── lib/                      # Infraestructura y librerías
transversales
    │   ├── auth/                 # Lógica de servidor (server-
auth.ts, otp-service.ts)
    │   ├── email/                # Nodemailer y plantillas de
correo
    │   ├── firebase/             # Configuración de admin.ts y
client.ts
    │   └── utils.ts              # Utilidades de estilos CSS
    └── modules/                  # Módulos desacoplados por
dominio de negocio
        ├── aplicaciones/         # Catálogo y gestión de
aplicaciones
        ├── auditoria/            # Registro de trazabilidad y
eventos
        ├── auth/                 # Componentes visuales de login y
doble factor
        ├── gestion-usuarios/     # Administración de usuarios,
roles y sedes
        ├── mis-aplicaciones/     # Dashboard de aplicaciones del
usuario
        ├── notas/                # Lógica de negocio y contexto de
Notas
        └── uikit/                # Showcases de componentes
visuales
```

12. Solución de Problemas Frecuentes (Troubleshooting)

Síntoma	Causa Raíz Probable	Solución Ver
Error al iniciar emuladores: `Java not found` o `JAVA_HOME not set`	Java no está instalado o no se encuentra en el `PATH`.	Instala Open Abre una nu

Error al iniciar emuladores: `Port 8080/9099/4000 already in use`	Otra instancia previa de Firebase o un proceso local retiene el puerto.	Identifica y f
		`NetTCPConn`
		`Process -F`

Síntoma	Causa Raíz Probable	Solución Ver
Error en login: `auth/invalid-api-key` en consola del navegador	Falta `NEXT_PUBLIC_FIREBASE_AUTH_EMULATOR_HOST` en `.env.local` o el servidor no se reinició.	Comprueba `NEXT_PUBL` **Reinicia Ne**
El emulador no conecta con Firestore:	`FIRESTORE_EMULATOR_HOST` no está disponible en el servidor.	Confirma qu `Emulator l` `.env.loca` `FIRESTORE`
El código de verificación OTP no llega:	Mailpit no está corriendo o el puerto SMTP es incorrecto.	Arranca Mail `.env.loca` http://localh
Error 403: `Tu usuario no está registrado en el sistema`	El usuario existe en Auth pero no tiene documento asociado en Firestore.	Ejecuta `nod` documento
Error 403: `No tienes permisos para acceder a la aplicación Notas`	El usuario no posee una asignación activa para la aplicación `"notas"` en `sedes[].asignaciones`.	El usuario se la asignació asignarle la
Fallo en `npm install`: `peer dependency conflict`	Restricciones estrictas en metadatos de paquetes secundarios con React 19.	Ejecuta siem `legacy-pee`
Los datos desaparecen al apagar la terminal:	Los emuladores operan por defecto en memoria volátil.	Utiliza `fire` `import=./e`

13. Checklist de Primera Ejecución para Nuevos Desarrolladores

- ☐ **Node.js verificado:** `node -v` devuelve `v22.x`.
- ☐ **npm verificado:** `npm -v` devuelve `10.x`.

- [ ] **Java verificado:** `java -version` responde sin errores (JDK 11+ o 21).
- [ ] **Firebase CLI verificado:** `firebase --version` devuelve `15.x`.
- [ ] **Mailpit verificado:** `mailpit version` responde correctamente.
- [ ] **Dependencias instaladas:** `npm install --legacy-peer-deps` o `npm ci --legacy-peer-deps` completado con éxito.
- [ ] **Archivo `.env.local` preparado:** Copiado desde `.env.example` con los hosts apuntando a `127.0.0.1`.
- [ ] **Emuladores de Firebase iniciados:** `npm run emulators` activo; panel visible en http://localhost:4000.
- [ ] **Mailpit iniciado:** `mailpit` activo; bandeja visible en http://localhost:8025.
- [ ] **Datos iniciales sembrados:** `node scratch/seed-superadmin.mjs` ejecutado con éxito.
- [ ] **Next.js iniciado:** `npm run dev` corriendo; interfaz accesible en http://localhost:3000.
- [ ] **Inicio de sesión en SSO probado:** Autenticación probada contra los emuladores locales.
- [ ] **Código OTP recibido e ingresado:** Verificado en http://localhost:8025 y procesado en `/doble-factor`.
- [ ] **Acceso a Notas probado:** Ingreso en http://localhost:3000/login-notas verificado.
- [ ] **Chequeo de tipos ejecutado:** `npx tsc --noEmit` completado con 0 errores.